Sistema de Gestão de RH - Enterprise Edition

Sistema completo de gestão de recursos humanos (RH) com backend ASP.NET Core e frontend React, focado em folha de pagamento quinzenal (Ciclos: Dia 20 e Dia 05) com cálculos CLT brasileiros.

🎯 Funcionalidades

Gestão de Funcionários

- ☑ Cadastro completo com CPF, cargo, departamento
- ☑ Controle de admissão e demissão
- ☑ Gestão de usuários e níveis de acesso (RBAC)
- ☑ Histórico de alterações salariais

Controle de Frequência

- ☑ Lançamento individual e em lote
- ☑ Registro de faltas, horas extras (50%) e dobras (100%)
- ☑ Busca por ciclo de pagamento
- ☑ Notas e observações por registro

Folha de Pagamento

- ☑ Cálculos automáticos por ciclo (Dia 20 e Dia 05)
- ☑ INSS progressivo conforme tabela CLT 2024/2025
- ☑ FGTS provisionado (8%)
- ☑ Desconto de faltas (Salário/20 por dia)
- ☑ Adicionais de horas extras
- ☑ Processamento em lote
- ☑ Holerite digital detalhado

Dashboards e Relatórios

- ☑ Dashboard RH com KPIs e estatísticas
- ☑ Dashboard Gerente com analytics
- ☑ Dashboard Funcionário com holerites
- ☑ Relatórios financeiros consolidados

🛠 Tecnologias Utilizadas

Backend

- **ASP.NET Core 8.0+** - Framework web
- **Entity Framework Core** - ORM
- **SQLite** - Banco de dados (dev)
- **JWT** - Autenticação
- **BCrypt.NET** - Hash de senhas
- **Swagger** - Documentação de API

Frontend ⭐ NOVO

- **React 19** - UI Library
- **Vite 7** - Build tool ultra-rápido
- **TanStack Query** - Server state management
- **Zustand** - Client state (leve e performático)
- **React Hook Form** - Formulários com validação
- **Zod** - Schema validation type-safe
- **Axios** - HTTP client com interceptors
- **Tailwind CSS** - Utility-first styling
- **Recharts** - Gráficos e visualizações
- **Lucide React** - Ícones modernos (600+)

📋 Pré-requisitos

- **.NET SDK 8.0 ou superior**
- **Node.js 18 ou superior**
- **npm ou yarn**

🚀 Instalação e Configuração

1. Clone o Repositório

```
git clone https://github.com/pikulitomarkin/sistemagestaorh.git
cd sistemagestaorh
```

2. Backend

```
cd backend

# Restaurar dependências
dotnet restore

# Aplicar migrações do banco de dados
dotnet ef database update

# Ou criar banco do zero
dotnet ef migrations add InitialCreate
dotnet ef database update

# Executar
dotnet run
```

O backend estará disponível em `http://localhost:5000`

Frontend

```
cd frontend

# Instalar dependências
npm install

# Configurar ambiente
cp .env.example .env
# Editar .env: VITE_API_URL=http://localhost:5000/api

# Executar em modo desenvolvimento
npm run dev

# Build para produção
npm run build
```

O frontend estará disponível em `http://localhost:5173`

Documentação Frontend Completa:

- ARQUITETURA.md - Documentação técnica (400+ linhas)
- ESTRUTURA.md - Resumo visual da estrutura
- INSTALACAO.md - Guia de instalação detalhado
- INDICE.md - Navegação rápida pela documentação

Usuários de Teste

RH

- **Username:** rh_test
- **Password:** 123
- **Acesso:** Gestão completa de funcionários, frequência e folha

Gerente

- Criar via interface RH

Funcionário

- Criar via interface RH

Estrutura do Projeto

```
sistemagestaorh/
├── backend/                  # ASP.NET Core 8.0
│   ├── Controllers/          # Endpoints da API
│   │   ├── AuthController.cs
│   │   ├── EmployeesController.cs
│   │   ├── AttendanceController.cs
│   │   └── PayrollController.cs
│   ├── Models/               # Entidades do banco
```

```
│   │   ├── User.cs
│   │   ├── Employee.cs
│   │   ├── Attendance.cs
│   │   └── Payroll.cs
│   ├── Services/            # Lógica de negócio
│   │   ├── INSSService.cs
│   │   ├── PayrollCalculationService.cs
│   │   └── PayrollService.cs
│   └── Program.cs           # Configuração e startup
│
├── frontend/                 # ⭐ NOVO: React 19 + Vite
│   ├── src/
│   │   ├── components/      # Componentes reutilizáveis
│   │   │   ├── ui/         # Design System (8 componentes)
│   │   │   ├── layout/     # DashboardLayout responsivo
│   │   │   └── payroll/    # Holerite Digital
│   │   ├── pages/          # Páginas principais
│   │   │   ├── auth/       # Login com validação
│   │   │   └── rh/         # Dashboard + Frequência
│   │   ├── services/       # API client (Axios + JWT)
│   │   ├── stores/         # Estado global (Zustand)
│   │   ├── hooks/          # Custom hooks
│   │   └── lib/            # Utilitários + Cálculos CLT
│   ├── 📖 ARQUITETURA.md   # Documentação técnica completa
│   ├── 📦 ESTRUTURA.md     # Resumo visual
│   ├── 🚀 INSTALACAO.md    # Guia de instalação
│   └── 🗺️ INDICE.md        # Navegação rápida
│
└── AUDITORIA_REFATORACAO.md  # Relatório técnico completo
```

🔒 Segurança

- ☑ Autenticação JWT com Bearer tokens
- ☑ RBAC (Role-Based Access Control) em todos os endpoints
- ☑ Senhas hasheadas com BCrypt
- ☑ CORS configurado
- ☑ HTTPS Redirection
- ☑ Validação de input no backend e frontend
- ☑ Auditoria de ações (CreatedBy, CreatedAt)

⚠ **Nota**: Atualize os pacotes JWT antes de produção (warnings NU1902)

📊 Lógica de Negócio - Folha de Pagamento

Ciclos de Pagamento

Ciclo 1 (Dia 20)

- **Período**: Dia 05 ao dia 19 do mês
- **Pagamento**: 50% do salário base + adicionais - descontos
- **INSS**: Não descontado (provisão)

- **Exemplo**: Janeiro/2026 → 05/01 a 19/01 → Pago em 20/01

Ciclo 2 (Dia 05)

- **Período**: Dia 20 do mês anterior ao dia 04 do mês atual
- **Pagamento**: 50% do salário base + adicionais - descontos - INSS
- **INSS**: Descontado integralmente (cálculo sobre salário mensal)
- **Exemplo**: Janeiro/2026 → 20/12/2025 a 04/01/2026 → Pago em 05/01

Cálculos

INSS Progressivo (Tabela 2024/2025)

Faixa Salarial	Alíquota
Até R$ 1.412,00	7,5%
R$ 1.412,01 a R$ 2.666,68	9%
R$ 2.666,69 a R$ 4.000,03	12%
R$ 4.000,04 a R$ 7.786,02	14% (teto)

FGTS

- **Alíquota**: 8% sobre salário bruto
- **Provisionado** mensalmente (não descontado do funcionário)

Faltas

- **Cálculo**: `(Salário Mensal / 20) × Dias de Falta`
- **Aplicado**: Em ambos os ciclos proporcionalmente

Horas Extras

- **50%**: `Valor Hora × 1,5 × Horas`
- **100% (Dobras)**: `Valor Hora × 2 × Horas`
- **Valor Hora**: `Salário Mensal / Horas Mensais (220)`

🧪 Testes

```
# Backend
cd backend
dotnet test

# Frontend
cd frontend
npm run test
```

⚠ **Nota**: Testes automatizados ainda não implementados. Ver AUDITORIA_REFATORACAO.md

🚀 Deploy

Railway (Recomendado)

1. Conecte seu repositório GitHub
2. Configure variáveis de ambiente
3. Deploy automático em cada push

Azure

1. Crie App Service para backend
2. Crie Static Web App para frontend
3. Configure connection strings

Render

1. Web Service para backend
2. Static Site para frontend

🐛 Problemas Conhecidos

Ver arquivo `AUDITORIA_REFATORACAO.md` para lista completa de:

- ☑ Problemas corrigidos
- ⚠ Vulnerabilidades conhecidas
- 📋 Melhorias recomendadas

📝 Changelog

v2.0.0 (24/01/2026) - Refatoração Enterprise

- ☑ RBAC completo implementado
- ☑ Precisão decimal corrigida
- ☑ Sistema de design enterprise
- ☑ Controllers com CRUD completo
- ☑ Tratamento robusto de exceções
- ☑ Componentes UI reutilizáveis
- ☑ Toast notifications
- ☑ Loading states
- ☑ Auditoria de ações

v1.0.0 - Versão Inicial

- Autenticação básica
- Gestão de funcionários
- Cálculos de folha

🤝 Contribuição

1. Fork o projeto
2. Crie uma branch para sua feature (`git checkout -b feature/AmazingFeature`)
3. Commit suas mudanças (`git commit -m 'Add some AmazingFeature'`)
4. Push para a branch (`git push origin feature/AmazingFeature`)
5. Abra um Pull Request

📄 Licença

Este projeto está sob a licença MIT.

👥 Autores

- **Marco** - Desenvolvimento inicial
- **CTO/Lead Designer** - Auditoria e refatoração enterprise (24/01/2026)

📞 Suporte

Para suporte, abra uma issue no GitHub ou entre em contato com a equipe de desenvolvimento.

⚠️ **IMPORTANTE**: Antes de usar em produção, leia o arquivo `AUDITORIA_REFATORACAO.md` para entender todas as melhorias implementadas e próximos passos recomendados.